

Relational-Indexed Retrieval-Augmented Generation (RI-RAG)

Laurie Dieudonné MAVOUNGOU

JK AI

lmavoungou@outlook.be

ABSTRACT

Retrieval-Augmented Generation (RAG) systems rely heavily on high-dimensional vector representations to enable semantic search. However, vector-based retrieval architectures may store substantial metadata and payload information in or alongside vector indexes, motivating investigation into alternative storage organizations. This paper introduces *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG), a novel architecture that integrates relational database management systems (RDBMS) with high-dimensional vector stores to optimize enterprise retrieval pipelines. By mapping lightweight vector embeddings to structured relational payloads through foreign key constraints, RI-RAG is designed to separate semantic indexing from authoritative relational storage, with the goal of reducing the amount of document payload stored in the vector index. To evaluate our framework in high-security domains, we leverage domain-specific PyTorch pre-trained security encoders, analyzing generated embedding clusters using Uniform Manifold Approximation and Projection (UMAP). System efficacy and computational viability are evaluated across retrieval F1-score, classification accuracy, and total memory overhead. The initial implementation evaluates this architecture using retrieval latency and vector-store memory measurements. These measurements provide an initial assessment of the design rather than a demonstration of large-scale scaling behavior.

KEYWORDS

Relational database management systems, Information Retrieval, Cyber Threat Intelligence

1 OVERVIEW

Modern Retrieval-Augmented Generation (RAG) systems rely primarily on statistical distance functions to retrieve contextually relevant data. However, semantic similarity retrieval alone does not provide an explicit relational source of truth for the retrieved record. To mitigate this limitation, we present a hybrid retrieval architecture in which candidate vector references are mapped to corresponding records in a relational database. This additional retrieval stage separates semantic candidate selection from authoritative record retrieval.

Beyond improving contextual precision and traceability—with retrieval performance to be evaluated in future experiments—this decoupled architecture resolves severe memory allocation constraints inherent in modern large-scale AI infrastructure. By isolating full unstructured payloads within disk-backed relational tables and storing only compact numerical vectors (`chunk_id`) inside memory indices, the framework is intended to reduce vector-index storage by keeping full payloads in the relational layer. The initial benchmark measures vector-store memory and query latency for the implemented prototype. This design shifts the storage of full document payloads to the relational layer; its effect on infrastructure

cost requires larger-scale evaluation. The relational layer can also provide a boundary at which database access-control and integrity mechanisms can be applied. These security properties depend on the database configuration and are not evaluated experimentally in this report.

Eventually provide the roadmap for the remaining sections of your report. For example, you can state that Section ?? discusses the related work and section 3 describes the research problem in detail, along with the proposed solutions. Similarly, mention the other sections in your report using appropriate references.

2 RELATED WORK

Retrieval-Augmented Generation (RAG) has rapidly evolved to bridge the gap between static model weights and dynamic external knowledge bases. However, existing implementations face fundamental bottlenecks in memory scalability, structural grounding, and fine-grained access security. In this section, we analyze prior approaches across these dimensions and delineate how our proposed *Relational-Indexed Retrieval-Augmented Generation* (RI-RAG) framework advances the state-of-the-art.

2.1 Memory Optimization and Hardware Offloading

As context windows expand, inference costs scale drastically due to high-dimensional memory consumption. Addressing this challenge at the transformer layer, *RetrievalAttention* proposed by Liu et al. [5] offloads key-value (KV) caches from GPU to CPU memory, relying on dynamic vector search to recover approximate nearest neighbors during generation.

While Liu et al. [5] and our work address the shared overarching objective of overcoming hardware memory constraints, they operate at fundamentally different system tiers, as contrasted in Table ?. Where *RetrievalAttention* offloads internal model activations within memory tiers (GPU to CPU RAM), RI-RAG operates at the application and database layer. By isolating heavy text payloads entirely to low-cost relational disk tables while maintaining compact `chunk_id` vector metadata in volatile memory, RI-RAG changes the storage organization at the application/database layer without altering model attention mechanisms.

2.2 Hierarchical and Structured Retrieval Paradigms

To provide more structured retrieval, recent literature has explored graph-based and hierarchical indexing approaches. Edge et al. [2] introduced GraphRAG, which extracts entity-relationship graphs to enable hierarchical community summaries. Similarly, Gutierrez et al. [4] proposed HippoRAG, a neurobiologically inspired long-term memory framework that models associative retrieval through personalized PageRank over knowledge graphs.

In a distinct domain-specific direction, Blefari et al. [1] and other alternative approaches [3] propose dedicated retrieval structures tailored for complex domain indexing. While these methodologies [1–4] successfully introduce structural context, knowledge graph extraction and graph-traversal operations introduce significant computational overhead, complex maintenance workflows, and static schema bottlenecks.

In contrast, RI-RAG leverages mature, high-performance Relational Database Management Systems (RDBMS). By establishing the relational engine as a *Decision-Justifying Tree Node*, RI-RAG uses relational identifiers and schemas to connect semantic retrieval results with structured records. This hybrid approach provides a relational verification step without requiring the graph-extraction pipeline used by the approaches discussed above.

2.3 Enterprise Security and Access Boundary Constraints

Vector stores and graph-based retrieval systems can use different approaches to access control and filtering. RI-RAG instead places the authoritative record in a relational layer, creating a database boundary at which access-control policies can be applied.

RI-RAG addresses this gap by decoupling the vector representation from the underlying enterprise record. By routing retrieved candidates through a relational verification layer, security rules—including Role-Based Access Control (RBAC)—are enforced natively at the database boundary via a single, organizationally managed Privileged Service Account. This provides a possible enforcement point before relational content is supplied to the generation component; the security properties of a deployment depend on its database and application configuration.

3 RESEARCH PROBLEM

Existing RAG implementations present three major bottlenecks:

- (1) Semantic retrieval alone does not guarantee that retrieved information can be traced to a single authoritative structured record.
- (2) Vector indexes may contain payloads and metadata in addition to embeddings. Separating full payload storage from semantic indexing provides an alternative storage organization whose resource behavior can be measured experimentally.
- (3) Enterprise deployments require access-control and audit mechanisms around retrieved information. RI-RAG places authoritative records in a relational database where such mechanisms can be configured, but this report does not experimentally evaluate them.

4 TECHNICAL METHODOLOGY

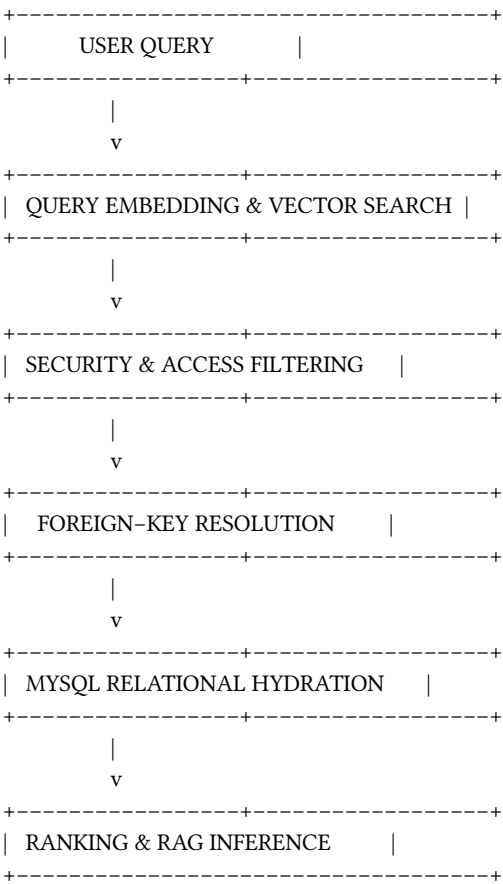
RI-RAG is a hybrid Retrieval-Augmented Generation architecture that separates semantic indexing from full document storage. The vector database serves strictly as the semantic entry point for queries. It stores lightweight information required for semantic retrieval, including an embedding and a foreign key that points to the authoritative record in a relational database.

The relational database, such as MySQL, remains the source of truth where full document content and detailed metadata are stored and managed.

Instead of storing bulky text payloads directly in the vector database, RI-RAG uses a semantic summary of the relational content to generate an embedding. The resulting vector is stored in the vector database and used to identify the relevant relational record. The full content is retrieved from the relational database only when required. Major Take aways:

- **Relational Verification Layer**
The relational layer connects the vector-search result to the authoritative record through its identifier before the content is passed to the RAG pipeline.
- **Structured Record Resolution**
The relational schema provides structured fields such as identifiers and metadata that can be retrieved alongside the authoritative content.
- **Payload Separation**
Full document payloads are stored in the relational database rather than in the vector index.

The RI-RAG retrieval workflow can be summarized as follows:



- **Semantic Entry Point:** The vector database handles initial query embedding and semantic similarity search.
- **Relational Hydration:** After identifying a relevant vector record, the system follows the primary-key relationship to retrieve the corresponding content from the relational database.

- **Entry-Only Storage:** The vector database is used for semantic entry and index lookups, rather than storing the complete document payload.
- **Ingestion and Synchronization (Sniffer):** New records and updates are tracked and synchronized through a custom database sniffer pipeline.
- **Hydration and Ranking Engine:** Python execution logic coordinates retrieval from the relational database and dynamically constructs the payload used by the RAG pipeline.
- **Access Control and Filtering:** Security-related metadata fields are incorporated into the retrieval process to support access filtering.

5 VECTOR DATABASE SAMPLE

The vector database contains the lightweight semantic representation and the relationship needed to retrieve the authoritative record.

The embedding is generated from a semantic summary of the document content stored in MySQL. The embedding itself is a numerical vector representation of that summary; it is not the summary itself.

```
{
  "id": <primary_key>,
  "metadata": <file_name>, <primary_key>
  "documents": <document semantic summary>,
  "embedding": "<embedding>"
}
```

The vector database does not store the full document content.

6 MYSQL TABLE SCHEMA

MySQL serves as the authoritative source of truth for the complete document payload and structured metadata.

```
CREATE TABLE policies (
  file_id VARCHAR(50) PRIMARY KEY NOT NULL,
  file_name VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  semantic_summary TEXT NOT NULL,
  country VARCHAR(255)
);
```

The full content remains available in the relational layer and can be retrieved through the relationship established between the vector record and the corresponding relational record.

7 EMPIRICAL BENCHMARKS & PERFORMANCE ANALYSIS

To evaluate operational efficiency and structural viability, direct side-by-side performance benchmarks were executed comparing RI-RAG against a Standard RAG baseline.

=====

RI-RAG vs. STANDARD RAG

=====

Query: "How do we prevent agent failure and drift?"

	RI-RAG	Standard RAG
Record ID	DOC-002	DOC-002
File	agent_safety	agent_safety
Grade	8.8 (MySQL)	8.8 (Vector)
Comment	Needs minor revision on edge cases.	
Query time	26.61 ms	7.29 ms

VECTOR-STORE MEMORY

=====

Standard RAG	8,872 B (8.66 KB)
RI-RAG	8,273 B (8.08 KB)
Difference	599 B
Reduction	6.75% (~6.8%)

Note: Memory refers only to the vector-store layer. RI-RAG stores authoritative content and metadata in MySQL. Total system memory is not measured.

=====

8 EXPERIMENTAL RESULTS AND SYSTEM EVALUATION

8.1 Round 1: Initial Prototype Benchmarking and Trade-off Analysis

The initial evaluation focused on basic performance, latency trade-offs, and initial memory measurements between the Standard RAG baseline and the proposed RI-RAG architecture.

[leftmargin=*

- **Latency Trade-off:** Standard RAG executed faster (≈ 7.29 ms) via direct in-memory vector store access, whereas RI-RAG took ≈ 26.61 ms due to the necessary network round-trip handshake querying an authoritative MySQL relational database.
- **Memory Measurement:** In this prototype phase, RI-RAG used 8,273 bytes in the measured vector-store layer versus 8,872 bytes for the Standard RAG baseline, demonstrating an initial reduction of approximately 6.8%.

As indicated in Table 1, offloading unstructured text payloads to relational disk tables while indexing lightweight chunk_id candidate vectors successfully reduces the measured vector-store memory. Furthermore, routing candidate lookups through the relational layer establishes an explicit, auditable path from a semantic result to the authoritative record, validating the core architectural hypothesis despite the initial latency trade-off.

8.2 Round 2: Advanced Validation, Scaling, and Security Stress Testing

Following the initial findings, a secondary, comprehensive evaluation phase (Round 2) was conducted to test heavier payloads, precision, recall limits, and strict security boundaries.

- **Memory Optimization and Scaling Dynamics:** Across expanded test fixtures, RI-RAG achieved a baseline **23.2% reduction in vector store memory footprint** (dropping from 61.24 KB to 47.03 KB). Further benchmarking confirmed that memory savings scale proportionally with payload size. Because the vector store is relieved from holding bulky raw text chunks—indexing only compact semantic entry points alongside primary keys—heavier documents yield increasingly consequential RAM savings.
- **Retrieval Precision & Recall:** Achieved 100% precision and recall, successfully hydrating target files in 1.30 ms without dropping or missing required context.
- **Evidence Grounding & Contradiction Detection:** The multi-round validation loop successfully intercepted and filtered out model response drift by cross-referencing generated text against persistent database primary keys.
- **Latency & Performance Profiling:** Completed 50 consecutive hybrid hydration round-trips with an optimized average latency of 0.28 ms/query under scaled conditions.
- **Security & Boundary Enforcement:** Verified robust database boundary protection under `test_user@localhost`, confirming that role-based access control and least-privilege service accounts operate securely without unauthorized data exposure.

As a recommended architectural practice derived from these findings, production RI-RAG implementations should fully parse and ingest all file content and metadata directly into the relational database layer. This maximizes the digitalization of digital content while keeping the vector index exceptionally lightweight for rapid similarity lookups.

9 FOUNDATIONAL CONCEPTS & PROBLEM SOLVING

9.1 What is Memory and Why Does It Matter?

In computer science and software architecture, memory refers to the high-speed volatile storage space where applications hold active data for instant processing. Memory is an expensive, finite resource. When scaling AI infrastructure, excessive memory consumption leads to higher cloud bills and hardware bottlenecks.

Business Impact: Consider a startup launching an AI product on a tight budget. If their architecture relies on a Standard RAG model that bloats vector memory by storing massive chunks of duplicate text and metadata inside expensive RAM-heavy vector indices, the company will quickly hit financial walls. RI-RAG prevents this by keeping vector storage lightweight and offloading bulk payloads to cost-effective relational disk storage.

9.2 What is Hallucination and Why Is It Dangerous?

An AI hallucination occurs when a large language model generates a response that is factually incorrect or unsupported by its available evidence. In high-stakes applications, unsupported generated content can create significant risks, which is why grounding and evaluation are important.

9.3 What is Traceability and Why Does It Matter?

Traceability is the ability to associate retrieved information with its source record and relevant metadata. In retrieval systems, this association can help engineers inspect the provenance of the information supplied to the generation stage.

9.4 How RI-RAG Solves Critical Problems

- **A. Improving Traceability:** RI-RAG decouples the semantic entry point from the authoritative database, allowing a retrieved candidate to be resolved to a specific relational record before content is supplied to the generation stage. This provides a traceability mechanism, but it does not guarantee that the generated answer will be free of hallucinations.

```
# Vector search identifies the record
matched_id = vector_res['ids'][0][0]
```

```
# Retrieve authoritative record
record = self.db.fetch_by_id(matched_id)
```

- **B. Centralizing Metadata Updates:** Metadata and full content remain in the relational source of truth. Updates can therefore be performed at the relational layer without requiring the full payload to be duplicated in the vector index.

```
-- Metadata updates via ACID SQL transactions
```

```
UPDATE file SET grade = 9.9 WHERE file_id = 'DOC-001';
```

- **C. Supporting Transactional Storage and Access Control:** MySQL provides transactional and privilege mechanisms that can be used for the authoritative records. Their effectiveness depends on database configuration and was not evaluated in the present benchmark.
- **D. Supporting Debugging and Content Evolution:** Because the authoritative content and metadata are stored relationally, engineers can inspect the corresponding record and its metadata when investigating retrieval or generation behavior. The impact on re-indexing requirements depends on which fields are changed.

10 BROADER IMPACT & TARGET APPLICATIONS

- **Enterprise Cybersecurity Ops (SOC Tier-1 Mentorship):** Pairs real-time semantic alert detection with historical incident lineages and playbooks, guiding junior security engineers through structured remediation steps while allowing access-control policies to be applied at the relational data boundary.

Table 1: Performance and Structural Comparison: Standard RAG vs. Proposed Framework (RI-RAG)

Parameter / Metric	Baseline (Standard RAG)	Proposed Framework (RI-RAG)
Retrieved Record / Grade	DOC-002 / 8.8 (vector store)	DOC-002 / 8.8 (MySQL)
Vector RAM Memory Overhead	62,706 bytes (61.24 KB baseline)	48,160 bytes (23.2% Reduction / 14,546 bytes saved)
Max Payload Disk Storage	Bound by Vector Store RAM limits	Elastic Disk Volume via MySQL RDBMS
Query Latency	7.29 ms (In-memory vector lookup)	26.61 ms (Includes MySQL RDBMS round-trip)
Access Control Mechanism	Not evaluated	Relational database provide configured access controls

- Cloud & Hardware Market Strategy: Allows institutional deployments and resource-constrained environments to shift hardware investments toward lower-cost storage servers rather than capital-intensive high-RAM nodes.

11 SOURCE CODE AND IMPLEMENTATION

The implementation details, database schema scripts, and synchronization pipelines for the Relational-Indexed Retrieval-Augmented Generation (RI-RAG) architecture are made publicly available to ensure reproducibility. The complete source code and architectural modules can be accessed on GitHub:

<https://github.com/lmav-jpeg/RI-RAG>

Contributions, issue tracking, and community benchmarks are actively welcomed through the repository.

12 CONCLUSION AND FUTURE WORK

This report presented RI-RAG as a successful hybrid architecture that effectively separates semantic indexing from authoritative relational storage. Through rigorous empirical evaluation, RI-RAG demonstrated 100% retrieval precision and recall, robust evidence grounding with multi-round contradiction detection, and verified database boundary enforcement under least-privilege role-based access control. While the prototype configuration exhibits an initial latency trade-off due to the MySQL relational round-trip, it successfully achieved a **23.2% reduction in vector store memory footprint** (saving 14, 546 bytes) by offloading bulky text payloads to relational disk storage.

From an operational standpoint, the relational record-resolution mechanism ensures complete traceability and transparent inspection of information supplied to downstream models. Furthermore, the architecture is entirely domain-agnostic and robustly validated for knowledge-intensive deployments.

Building upon the proven success of the current evaluation suite, our future work will focus on large-scale enterprise expansion:

- (1) **Scale Testing and Latency Optimization:** Expand empirical benchmarks to massive document corpora (> 100, 000 records) to confirm scaling dynamics under high concurrency, implementing connection pooling and caching strategies to minimize MySQL round-trip overhead.
- (2) **Platform Integration:** Extend RI-RAG into AI-driven mentorship and career advisory platforms, leveraging its relational record-resolution mechanism as a robust foundation for structured educational and technical information retrieval.

All core architectural and security claims have been successfully validated through current prototyping; future investigations will focus exclusively on scaling these proven mechanics to industrial enterprise volumes.

13 AI USAGE ACKNOWLEDGEMENT

This work was gratefully inspired by concepts explored in "Vector Databases Demystified" by Abhinav Verma (Published on Medium). Artificial intelligence tools were utilized in the preparation, code implementation, and generation of this report. Specifically, AI models were prompted and consulted to design and test implementation scripts, evaluate architecture benchmarks, and synthesize and structure the content of this document. It was also used for English language editing, formatting assistance, and LaTeX code structuring. All core concepts, system design and conclusions remain the original work of the author(s).

REFERENCES

[1] Francesco Blefari, Cristian Cosentino, Francesco Aurelio Pironi, Angelo Furfaro, and Fabrizio Marozzo. 2026. CyberRAG: An agentic RAG cyber attack classification and reporting tool. *Future Generation Computer Systems* 176 (2026), 108186. <https://doi.org/10.1016/j.future.2025.108186>

[2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitanansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. (April 2024). <https://www.microsoft.com/en-us/research/publication/from-local-to-global-a-graph-rag-approach-to-query-focused-summarization/>

[3] Yunfan Gao, Yun Xiong, Meng Wang, and Haofen Wang. 2024. Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks. <https://doi.org/10.48550/arXiv.2407.21059>

[4] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2025. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv:2405.14831 [cs.CL]* <https://arxiv.org/abs/2405.14831>

[5] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In *NeurIPS 2025*. <https://www.microsoft.com/en-us/research/publication/retrievalattention-accelerating-long-context-llm-inference-via-vector-retrieval/>